



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escola Tècnica Superior d'Enginyeria Informàtica
Universitat Politècnica de València

Manual de Configuración e Instalación

Trabajo Fin de Grado

Grado en Ingeniería Informática

Autor: Lourdes Francés Llimerá

Tutor: Tutor/a

2025-2026

Tabla de contenidos

Tabla de contenidos.....	2
1. Introducción.....	3
2. Fase 1 – Máquina Virtual (VMware + Ubuntu).....	3
2.1. Instalar VMware Workstation Pro	3
2.2. Descargar Ubuntu 22.04.4 LTS	3
2.3. Crear y configurar la máquina virtual.....	3
2.4. Solucionar el warning de audio de VMware	3
3. Fase 2 – ROS 2 Humble	4
3.1. Prerrequisitos	4
3.2. Configurar locale.....	4
3.3. Añadir repositorio de ROS 2	5
3.4. Instalar ROS 2 Desktop	5
3.5. Verificar la instalación	5
4. Fase 3 – TurtleBot3 y workspace.....	6
4.1. Instalar paquetes TurtleBot3	6
4.2. Clonar y compilar el workspace (laric_ws)	6
4.3. Prueba del simulador.....	6
5. Fase 4 – Framework RAI	7
5.1. Instalar el Framework RAI modificado.....	7
5.2. Verificar RAI	7
6. Fase 5 – Entorno virtual Python (laric_env).....	8
6.1. Crear y activar el entorno virtual	8
6.2. Instalar dependencias LangChain y audio.....	8
6.3. Configurar la API key de Groq.....	9
6.4. Compilar el paquete ROS.....	9
7. Primer arranque	9
7.1. Snapshot de la VM.....	10

1. Introducción

Este manual describe el proceso completo de preparación del entorno para ejecutar el proyecto, desde la instalación de la máquina virtual hasta el primer arranque del sistema. El entorno de desarrollo es Ubuntu 22.04 LTS corriendo en VMware Workstation Pro sobre Windows. Todos los pasos deben seguirse en orden.

2. Fase 1 - Máquina Virtual (VMware + Ubuntu)

2.1. Instalar VMware Workstation Pro

VMware Workstation Pro 17 es gratuito para uso personal desde que Broadcom adquirió la empresa. Para descargarlo:

1. Acceder al Portal de Soporte de Broadcom: support.broadcom.com.
2. Crear una cuenta si no se tiene una.
3. Navegar a: My Downloads > Free Software Downloads > [VMware Workstation Pro](#).
4. Descargar el instalador .exe para Windows y ejecutarlo con opciones por defecto.

2.2. Descargar Ubuntu 22.04.4 LTS

Descargar la imagen ubuntu-22-04.4-desktop-amd64 desde ubuntu.com. Guardar el archivo .iso en una carpeta local, no debe abrirse directamente.

2.3. Crear y configurar la máquina virtual

1. En VMware: File > New Virtual Machine > Typical.
2. Installer disc image file (iso) > Seleccionar la ISO descargada.
3. Completar nombre de usuario y contraseña en Easy Install.
4. Disk Capacity: Mínimo 50 GB, opción Split virtual disk.
5. Antes de pulsar Finish, click en [Customize Hardware...]:
 - Memory: Mínimo 4096 MB (8192 MB recomendado si el PC tiene 16 GB o más).
 - Processors: 1 procesador y mínimo 2 cores por procesador (4 recomendado).
 - Display: Activar Accelerate 3D graphics con 2 GB de memoria gráfica o la máxima recomendada por el sistema (sin la aceleración 3D, RViz no arranca correctamente).
 - Network Adapter: NAT.

2.4. Solucionar el warning de audio de VMware

VMware usa por defecto el driver de sonido es1371 (antiguo), que genera warnings de ALSA en Ubuntu. Para evitarlos:

1. Apagar la máquina virtual.
2. Abrir el archivo de configuración (.vmx) de la VM con el Bloc de Notas desde la carpeta donde está guardada la máquina virtual.
3. Añadir al final la línea: `sound.virtualDev = "hdaudio"`.
4. Guardar el archivo y cerrarlo.
5. Volver a encender la VM.

3. Fase 2 - ROS 2 Humble

3.1. Prerrequisitos

```
sudo apt update && sudo apt upgrade -y

sudo apt install git curl gnupg2 lsb-release python3-pip python3.10-venv terminator -y
```

Tabla 1. Prerrequisitos.

- `update / upgrade`: Actualiza el sistema base para evitar conflictos de compatibilidad.
- `git`: Necesario para clonar (descargar) repositorios de código desde GitHub.
- `curl`: Permite descargar archivos y claves de seguridad desde internet directamente en la terminal.
- `gnupg2`: Herramienta de criptografía necesaria para verificar las firmas de los paquetes que se van a instalar.
- `lsb-release`: Identifica la versión exacta de Ubuntu que se emplea para descargar la versión correcta de ROS 2.
- `python3-pip`: Gestor de paquetes oficial de Python. Es imprescindible para poder instalar más adelante todas las librerías de inteligencia artificial, procesamiento de voz y dependencias del proyecto (`requirements.txt`) que no vienen incluidas en el sistema operativo.
- `python3.10-venv`: Instala la herramienta oficial de Ubuntu necesaria para poder aislar y gestionar los entornos virtuales de Python (`venv`) en la versión 3.10 del sistema.
- `terminator`: Un emulador de terminal avanzado que permite dividir la pantalla en varios paneles. Es casi imprescindible en ROS, ya que normalmente es necesario tener varios procesos corriendo a la vez a la vista.

3.2. Configurar locale

```
sudo locale-gen en_US en_US.UTF-8

sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

export LANG=en_US.UTF-8
```

Tabla 2. Configuración locale.

- `locale / UTF-8`: ROS 2 es estricto con la codificación de caracteres. Se configura el sistema en inglés y en formato UTF-8 para asegurar que los registros (logs),

los mensajes de error y las herramientas de compilación funcionen correctamente y no fallen por leer caracteres extraños o acentos.

3.3. Añadir repositorio de ROS 2

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
-o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-
archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME)
main" \
| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

Tabla 3. Añadir repositorio de ROS 2.

- `curl (ros.key)`: Descarga la clave de seguridad oficial de ROS para garantizar que el software es legítimo, ya que Ubuntu no trae ROS 2 en sus tiendas de aplicaciones por defecto.
- `echo / tee`: Le indica al sistema operativo dónde están los servidores (repositorios) oficiales para descargar ROS 2 de forma segura.

3.4. Instalar ROS 2 Desktop

```
sudo apt update

sudo apt install ros-humble-desktop ros-dev-tools -y

echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc

source ~/.bashrc
```

Tabla 4. Instalación de ROS 2 Desktop.

- `ros-humble-desktop`: Instala la versión completa de ROS 2, incluyendo herramientas visuales como RViz (para ver al robot en 3D).
- `ros-dev-tools`: Instala herramientas de desarrollo necesarias para compilar código propio más adelante (como `colcon`).
- `source ~/.bashrc`: Hace que los comandos de ROS 2 estén disponibles cada vez que se abre una nueva terminal. Sin esto, la terminal no reconocería comandos como `ros2`.

3.5. Verificar la instalación

```
# Terminal 1: ros2 run demo_nodes_cpp talker

# Terminal 2: ros2 run demo_nodes_py listener
```

Tabla 5. Verificación de la instalación.

- `talker / listener`: Es el "Hola Mundo" de ROS 2. Comprueba que el sistema de comunicación interno (DDS) funciona. El nodo `talker` (C++) publica mensajes y el `listener` (Python) los escucha, demostrando que distintos programas pueden comunicarse entre sí en tiempo real.

Si la Terminal 2 recibe los mensajes, ROS 2 está instalado correctamente.

4. Fase 3 – TurtleBot3 y workspace

4.1. Instalar paquetes TurtleBot3

```
sudo apt install ros-humble-turtlebot3* -y

echo 'export TURTLEBOT3_MODEL=waffle_pi' >> ~/.bashrc

source ~/.bashrc
```

Tabla 6. Instalación de paquetes TurtleBot3.

- `ros-humble-turtlebot3*`: Descarga los modelos 3D, las reglas físicas y los parámetros de navegación del robot.
- `TURTLEBOT3_MODEL=waffle_pi`: Guardar esta variable en el archivo `.bashrc` es crucial para indicarle a las simulaciones exactamente qué chasis y sensores debe cargar, ya que existen varios modelos físicos del robot.

4.2. Clonar y compilar el workspace (laric_ws)

```
cd ~

git clone https://github.com/lourdesfll29-maker/LARIC-TFG.git

cp -r ~/LARIC-TFG/laric_ws ~/laric_ws

cd ~/laric_ws

colcon build

echo 'source ~/laric_ws/install/setup.bash' >> ~/.bashrc

source ~/.bashrc
```

Tabla 7. Clonación y compilación del workspace.

- `git clone`: Descarga una copia exacta del repositorio del proyecto desde GitHub, incluyendo la documentación, el código fuente y los mapas.
- `cp -r`: Copia la carpeta `laric_ws` completa (con todo su contenido) desde el repositorio descargado al directorio principal del usuario (`~`). Se realiza como medida de seguridad para garantizar que las rutas absolutas dentro de los scripts de lanzamiento funcionen correctamente, y para aislar los archivos generados durante la compilación (`build`, `install`) del control de versiones de Git.
- `colcon build`: Compila todo el código fuente que se encuentra dentro de la carpeta `src` del repositorio recién clonado.
- `source install/setup.bash`: Enlaza el espacio de trabajo ya compilado con el núcleo de ROS 2 para que el sistema reconozca los paquetes personalizados.

4.3. Prueba del simulador

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Tabla 8. Prueba del simulador.

- `ros2 launch`: Arranca Gazebo y carga el mundo 3D junto con el modelo del TurtleBot3. Verifica que los paquetes del simulador y el entorno virtual se

cargan correctamente, asegurando un mundo simulado funcional donde poder probar el código de forma segura antes de pasar al robot físico.

Si Gazebo abre con el TurtleBot3 situado dentro del modelo 3D, el entorno de simulación base funciona correctamente y está listo.

5. Fase 4 — Framework RAI

5.1. Instalar el Framework RAI modificado

```
pip3 install uv

cd ~ && git clone https://github.com/lourdesf1129-maker/rai.git

cd rai

sudo ~/.local/bin/uv pip install --system -e src/rai_core

sudo ~/.local/bin/uv pip install --system -e src/rai_s2s
```

Tabla 9. Clonación del repositorio de RAI.

- `pip3 install uv`: Instala un gestor de paquetes de Python de alta velocidad necesario para procesar los archivos de configuración del proyecto.
- `git clone`: Descarga la versión adaptada (Fork) del framework desde el repositorio de la autora, asegurando que se incluyan las modificaciones de código específicas requeridas para este TFG.
- `uv pip install --system`: Compila e instala los paquetes directamente en las librerías del sistema operativo. Esto es imprescindible para asegurar que los nodos globales de ROS 2 puedan localizar y comunicarse con los módulos de RAI más adelante.
- El modificador `-e`: La bandera `-e` (editable) enlaza el código fuente local con el entorno de Python. Esto permite que cualquier modificación futura que se realice sobre los scripts del framework se aplique de forma instantánea sin necesidad de reinstalar el paquete..

5.2. Verificar RAI

```
python3 -c "from rai.communication.ros2 import ROS2Connector; print('RAI OK')"
```

Tabla 10. Verificación de RAI.

- `python3 -c`: Realiza un test rápido de una sola línea importando el conector principal de RAI. Si imprime "RAI OK" sin errores, confirma que Python detecta las librerías correctamente.

Al ejecutar este comando, es completamente normal que aparezcan dos mensajes de advertencia en la terminal:

1. Un aviso de `pydub` indicando que no se encuentra `ffmpeg`.
2. Un aviso del sistema indicando que `rai_interfaces` no está instalado.

Ambos mensajes deben ignorarse en este paso. El aviso de `ffmpeg` se solucionará automáticamente en el siguiente apartado (Fase 6) al instalar las dependencias de

audio del sistema, y el de `laric_interface` no afecta al núcleo de la simulación. Si al final de las advertencias la terminal imprime el mensaje RAI OK, la verificación ha sido un éxito.

6. Fase 5 — Entorno virtual Python (`laric_env`)

El agente requiere versiones específicas de LangChain que pueden entrar en conflicto con paquetes del sistema. Se usa un `virtualenv` con acceso al sistema para mantener la conexión con ROS 2.

6.1. Crear y activar el entorno virtual

```
cd ~/laric_ws

python3 -m venv --system-site-packages laric_env

source laric_env/bin/activate
```

Tabla 11. Creación y activación del entorno virtual.

- `venv`: Entorno virtual que actúa como una burbuja segura para instalar librerías de IA sin riesgo de generar conflictos de versiones con los paquetes de Ubuntu o ROS 2.
- `--system-site-packages`: Crea un entorno semi-permeable. Aísla las nuevas librerías de IA, pero permite que Python siga teniendo acceso a la instalación base de ROS 2 del sistema.

6.2. Instalar dependencias LangChain y audio

```
pip install -r requirements.txt

sudo apt install portaudio19-dev python3-pyaudio ffmpeg -y
```

Tabla 12. Instalación de dependencias LangChain y audio.

- `pip install -r requirements.txt`: Lee el archivo de configuración e instala de forma automatizada y en un solo bloque todas las librerías de Python necesarias (ecosistema LangChain, LangGraph, SpeechRecognition, PyQt5, entre otras), asegurando el despliegue de las versiones exactas compatibles con el proyecto.
- `portaudio19-dev` / `python3-pyaudio`: Paquetes de desarrollo del sistema operativo necesarios para compilar y permitir que los scripts de Python accedan físicamente al hardware de audio del ordenador (micrófono y altavoces).
- `ffmpeg`: Herramienta multimedia nativa de Ubuntu, imprescindible para que la librería de audio pueda realizar la conversión, compresión y lectura de flujos de voz (formatos `.mp3`, `.wav`) en tiempo real durante la interacción con el usuario.

Al ejecutar el comando `pip install`, es muy probable que la terminal muestre un bloque de advertencias de desinstalación (`Attempting uninstall...`) y un mensaje de error final en color rojo detallando conflictos de dependencias (`ERROR: pip's dependency resolver...`).

Este comportamiento es completamente normal y debe ignorarse. Ocurre porque el entorno virtual se creó con acceso a los paquetes globales (--system-site-packages). El instalador advierte que las versiones del sistema operativo difieren de las solicitadas, pero asegura el éxito aislando e instalando correctamente las versiones del archivo en el entorno local. Si la última línea confirma Successfully installed..., el proceso se ha completado correctamente.

6.3. Configurar la API key de Groq

Por motivos de seguridad y para evitar la exposición accidental de credenciales privadas en el repositorio, la clave de acceso al LLM se gestiona exclusivamente a través de una variable de entorno en el sistema operativo.

Para evitar tener que introducir la credencial manualmente cada vez que se abra una nueva terminal, se registra su configuración de forma permanente en el sistema ejecutando el siguiente bloque de comandos:

```
echo 'export GROQ_API_KEY=". . ."' >> ~/.bashrc
source ~/.bashrc
```

Tabla 13. Configuración de la API key de Groq.

Alternativamente, poner from_lab = True para usar el servidor Ollama del laboratorio UPV.

6.4. Compilar el paquete ROS

```
cd ~/laric_ws && source laric_env/bin/activate

colcon build --symlink-install --packages-select laric_core

source install/setup.bash
```

Tabla 14. Compilación del paquete ROS.

- colcon build --symlink-install: Compila el paquete de control por voz creando accesos directos a los scripts de Python en lugar de copias estáticas. Esto permite editar el código del agente de voz y probarlo inmediatamente sin tener que recompilar tras cada pequeño cambio.

7. Primer arranque

```
chmod +x ~/laric_ws/start_simulation.sh ~/laric_ws/stop_simulation.sh

cd ~/laric_ws && ./start_simulation.sh
```

Tabla 15. Primer arranque.

- chmod +x: Le da permisos de ejecución a los scripts para que Ubuntu los trate como programas y no como simples archivos de texto.

Esto lanza 4 procesos: Gazebo (10 s de espera) → Nav2+RViz (5 s) → HMI (3 s) → Agente (ventana de terminal visible). Antes de enviar comandos de movimiento o

navegación, es necesario realizar el '2D Pose Estimate' en RViz para inicializar la localización del robot. El sistema notificará en la interfaz cuando la localización se haya establecido.

7.1. Snapshot de la VM

Con todo funcionando, hacer en VMware: click derecho sobre la VM > Snapshot > Take Snapshot. Nombre recomendado: 'LARIC-OK'. Permite restaurar el entorno si algo se rompe.